

#AWSDevOps

#Ansible

#ConfigMgmt

#DevOps

#IaC

#YAML

Basic Interview Questions — Every Fresher Should Prepare

Q1. What is Ansible?

- Open-source IT automation tool developed by Red Hat.
- Used for configuration management, application deployment, and task automation.
- Agent-less: no software needs to be installed on managed nodes.
- Uses SSH (Linux) or WinRM (Windows) for communication.
- Written in Python; playbooks are written in YAML.

Q2. Why do companies use Ansible?

- Reduces manual effort — repetitive tasks are automated consistently.
- Agent-less architecture lowers operational overhead and security risk.
- Human-readable YAML makes it accessible to both Dev and Ops teams.
- Large module library covers cloud, OS, networking, and application layers.
- Supports multi-cloud and hybrid environments out of the box.

Q3. What problem does Ansible solve?

- Configuration drift: servers diverging from their intended state over time.
- Manual, error-prone setup of servers at scale.
- Inconsistency between dev, staging, and production environments.
- Slow, undocumented deployment processes that rely on individual knowledge.
- Ansible enforces idempotent, documented, repeatable infrastructure state.

Q4. What is Configuration Management?

- The practice of maintaining systems in a known, desired, and consistent state.
- Tracks and controls changes to software, hardware, and infrastructure configurations.
- Prevents configuration drift — systems deviating from their baseline over time.
- Ansible, Puppet, and Chef are common CM tools; Ansible is push-based and agent-less.
- In DevOps, CM enables immutable-style infrastructure and faster incident recovery.

Q5. How does Ansible work?

- The Control Node runs Ansible and holds playbooks, inventory, and roles.
- Managed Nodes are the target servers; they require no Ansible installation.
- Ansible connects to managed nodes via SSH and runs temporary Python modules.
- It reads the inventory file to know which hosts to target.
- Modules are pushed, executed, and then removed — leaving no persistent agent.
- Execution flow: `ansible-playbook` command → parse YAML → connect via SSH → run tasks → return results.

Q6. What is an Ansible Inventory?

- A file (static or dynamic) that lists the managed nodes Ansible will target.
- Default location: `/etc/ansible/hosts`; can be overridden with `-i` flag.
- Supports INI and YAML formats.
- Hosts can be grouped (e.g., `[webservers]`, `[databases]`) for selective targeting.
- Dynamic inventories pull host data from AWS EC2, GCP, Kubernetes, etc. at runtime.
- Variables can be scoped per host (`host_vars/`) or per group (`group_vars/`).

Note: Example: `[webservers]\n192.168.1.10\n192.168.1.11`

Q7. What is an Ansible Playbook?

- A YAML file that defines a set of automation tasks to run against an inventory.
- Consists of one or more 'plays' — each play targets a host group and runs tasks.
- More powerful than ad-hoc commands; supports variables, loops, conditionals, handlers.
- Playbooks are idempotent by design — re-running them should produce the same result.
- Entry point for most production Ansible usage; can call roles for modular structure.

Q8. What is a Task in Ansible?

- The smallest unit of work in a playbook — a single action to be performed.
- Each task calls an Ansible module (e.g., apt, yum, copy, template, service).
- Tasks are executed sequentially by default.
- Each task can have a 'name' for readability, a 'when' condition, 'register' to capture output, and 'notify' to trigger handlers.
- Example: Install nginx using the apt module on Ubuntu hosts.

Q9. What is a Role in Ansible?

- A structured, reusable way to organise playbook content into a standard directory layout.
- Standard directories: tasks/, handlers/, vars/, defaults/, files/, templates/, meta/.
- Roles promote separation of concerns — one role per application component.
- Can be shared via Ansible Galaxy (community role repository).
- Included in playbooks using the 'roles:' key or 'include_role' module.

Q10. Why do we use Roles?

- Reusability: write once, use across multiple playbooks and projects.
- Maintainability: changes to a role propagate wherever it is included.
- Readability: playbooks become high-level orchestration files instead of monolithic scripts.
- Collaboration: teams can own and version individual roles independently.
- Ansible Galaxy enables consumption of community-maintained, battle-tested roles.

Q11. What is YAML?

- YAML Ain't Markup Language — a human-readable data serialisation format.
- Uses indentation (spaces, not tabs) to define hierarchy.
- Supports scalars (strings, integers, booleans), lists, and mappings (key-value pairs).
- Files start with '---' (document start marker) optionally.
- Widely used in DevOps tooling: Ansible, Kubernetes manifests, Docker Compose, GitHub Actions.

Q12. Why does Ansible use YAML?

- YAML is readable by both humans and machines — reduces the learning curve.
- No need to learn a domain-specific language; existing Python/DevOps knowledge transfers.
- Aligns with the 'infrastructure as code' principle — configs can be version-controlled.
- YAML's structure maps naturally to Ansible's task-list and variable model.
- Reduces friction between Dev and Ops teams collaborating on automation.

Q13. What are Ansible Modules?

- Pre-built, reusable units of code that perform a specific action on a managed node.
- Examples: apt/yum (package management), copy/template (file management), service (service control), shell/command (run commands), aws_ec2 (cloud provisioning).
- Modules are idempotent — they check current state before making changes.

- Ansible ships with 3000+ built-in modules; custom modules can be written in Python.
 - Called within tasks using the module name as the task key.
-

Q14. What is the Command Module?

- Runs a single command on the remote node without invoking a shell.
 - Does NOT support shell features: pipes (`|`), redirects (`>`), variables (`$VAR`), `&&`.
 - Safer than the shell module — avoids shell injection risks.
 - Use when the command does not need shell interpretation.
 - Example use: reboot a server, run a binary, check a version with `--version`.
-

Q15. What is the Shell Module?

- Runs commands through the managed node's shell (`/bin/sh` by default).
 - Supports shell features: pipes, redirects, environment variables, glob expansion.
 - Less safe than command module — avoid when shell features aren't needed.
 - Use when you need to chain commands (`cmd1 | cmd2`) or use shell built-ins.
 - Can specify a different shell executable using the `'executable'` parameter.
-

Q16. What is `ansible.cfg`?

- The main configuration file for Ansible behaviour.
 - Ansible looks for it in: current directory → `~/.ansible.cfg` → `/etc/ansible/ansible.cfg` (in that order).
 - Key settings: inventory path, `remote_user`, `private_key_file`, `host_key_checking`, `roles_path`.
 - Allows project-level config overrides without changing system-wide defaults.
 - Disabling `host_key_checking` is common in dynamic environments but reduces security.
-

Q17. What is a Host Group?

- A named collection of hosts in the inventory file, targeted together in playbooks.
 - Defined in INI format with `[group_name]` or YAML format under `'children'`.
 - Allows running the same tasks on multiple servers in one play.
 - Groups can be nested — a parent group can contain child groups.
 - Special built-in groups: `'all'` (every host), `'ungrouped'` (hosts not in any group).
-

Q18. What are Variables in Ansible?

- Used to parameterise playbooks so they work across different environments.
 - Can be defined in: playbooks, inventory files, `host_vars/`, `group_vars/`, `roles/defaults`, `roles/vars`, extra vars (`-e` flag).
 - Variable precedence: extra vars (`-e`) have the highest priority; role defaults have the lowest.
 - Accessed using Jinja2 syntax: `{{ variable_name }}`.
 - Enables environment-specific configs (`dev/staging/prod`) from a single playbook.
-

Q19. What are Facts in Ansible?

- System information automatically gathered from managed nodes at the start of a play.
 - Collected by the `'setup'` module (runs implicitly when `gather_facts: true`).
 - Examples: `ansible_os_family`, `ansible_distribution`, `ansible_memtotal_mb`, `ansible_default_ipv4.address`.
 - Used in conditionals: `'when: ansible_os_family == "Debian"'` to branch task logic.
 - Can be disabled (`gather_facts: false`) to speed up playbook execution when not needed.
 - Custom facts can be added to `/etc/ansible/facts.d/` on managed nodes.
-

Q20. What is Ansible Vault?

- A built-in encryption feature to protect sensitive data in Ansible files.

- Encrypts files or individual string values using AES-256.
 - Common use: encrypting passwords, API keys, SSL certificates, database credentials.
 - Commands: `ansible-vault create`, `encrypt`, `decrypt`, `edit`, `view`, `encrypt_string`.
 - Vault password must be provided at playbook runtime: `--ask-vault-pass` or `--vault-password-file`.
 - Integrates with secrets managers (HashiCorp Vault, AWS Secrets Manager) for enterprise use.
-

Q21. What is the difference between Ansible and Terraform?

- Ansible: Configuration Management + Application Deployment — manages the STATE of existing infrastructure.
 - Terraform: Infrastructure Provisioning — creates and destroys infrastructure resources.
 - Ansible is procedural (how to do it); Terraform is declarative (what the end state should be).
 - Ansible does not maintain a state file; Terraform uses `terraform.tfstate` to track resources.
 - In practice, they complement each other: Terraform provisions EC2/EKS, Ansible configures them.
 - Ansible is agent-less (SSH); Terraform uses provider APIs (AWS, GCP, Azure).
-

Q22. What is the difference between Ansible and Shell Scripting?

- Idempotency: Ansible modules check current state and skip changes if already applied; shell scripts re-execute unconditionally.
 - Readability: Ansible YAML is self-documenting; shell scripts require comments to be understood.
 - Error handling: Ansible fails fast and reports clearly; shell scripts need explicit error handling (`set -e`, etc.).
 - Scalability: Ansible runs tasks in parallel across hundreds of nodes; shell scripts need manual parallelisation.
 - Maintainability: Ansible roles and variables make scaling complexity manageable; shell scripts become brittle at scale.
 - Shell scripts are faster to write for one-off tasks; Ansible is better for repeatable, team-owned automation.
-

Q23. Why is SSH important in Ansible?

- SSH is the default transport mechanism for Ansible to connect to Linux managed nodes.
 - Ansible pushes and executes temporary Python modules over SSH — no persistent agent needed.
 - SSH key-based authentication is the standard (preferred over password-based for security).
 - The control node must have SSH access (key pair + correct user) to all managed nodes.
 - `ansible.cfg` settings like `remote_user`, `private_key_file`, and `port` configure SSH behaviour.
 - SSH multiplexing (ControlMaster) can be enabled to speed up execution across many hosts.
-

Q24. Why is Ansible important in DevOps projects?

- Automates server provisioning and configuration in CI/CD pipelines post-Terraform.
 - Eliminates environment inconsistency — same playbook runs in dev, staging, and prod.
 - Integrates with Jenkins, GitHub Actions, and GitLab CI as a deployment step.
 - Enables rapid disaster recovery — re-run a playbook to restore a server to known state.
 - Supports zero-downtime deployments via rolling updates and `delegate_to` strategies.
 - Reduces dependency on tribal knowledge — automation is codified, version-controlled, and reviewable.
-

Q25. What are the advantages of Ansible automation?

- Agent-less: reduces attack surface and operational overhead on managed nodes.
 - Low learning curve: YAML + SSH knowledge is sufficient to get started.
 - Idempotent: safe to run repeatedly; only changes what is out of desired state.
 - Extensible: 3000+ modules plus custom Python modules for any requirement.
 - Community: Ansible Galaxy provides thousands of reusable, tested roles.
 - Cross-platform: manages Linux, Windows, network devices, cloud APIs, and containers.
 - Audit trail: playbooks in version control provide a clear history of infrastructure changes.
-

Pro Tip: For any Ansible question in interviews, anchor your answer to a real scenario. Example: 'In my ApnaKart project, I used Ansible to configure EC2 nodes post-EKS setup, ensuring consistent package versions across environments.' Practical context always scores higher than theory alone.